

M.Sc AI Part 1
CSI 5201 — Fundamentals of Natural Language Processing
Assignment

Student Name: __Kowsya Mutkundu__

Date of Submission: 02/09/2025

**Topic : A detailed case study of NLTK - A Natural Language
Toolkit for English language**

Dataset Description

The dataset contains records of students with attributes related to their daily habits and performance.

Installs the Python packages needed:

- **nltk** → Natural Language Toolkit for tokenization, POS tagging, etc.
- **scikit-learn** → ML models, vectorization, evaluation.
- **langdetect** → detects language (English/Hindi).
- **matplotlib**, **wordcloud** → visualizations.

```
# Run this cell if running on a fresh Colab runtime (it may take ~1-2 minutes)
!pip install -q nltk scikit-learn langdetect matplotlib wordcloud
# Optional: spaCy & Hindi/English models (large; run only if you want dependency parsing)
# !pip install -q spacy
# !python -m spacy download en_core_web_sm
# !python -m spacy download hi_core_news_sm
```

981.5/981.5 kB 20.6 MB/s eta 0:00:00

Preparing metadata (setup.py) ... done
Building wheel for langdetect (setup.py) ... done

Import Libraries + Download NLTK Data

```
import os, re, string
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# NLP
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize, regexp_tokenize
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer
from nltk import pos_tag, ne_chunk
from nltk.probability import FreqDist

# ML
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix, classification_report
from sklearn.metrics.pairwise import cosine_similarity

# Language detection
from langdetect import detect, DetectorFactory
DetectorFactory.seed = 0

# Download NLTK resources (safe to re-run)
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('maxent_ne_chunker')
nltk.download('words')

print("Imports and NLTK downloads done ✅")
```

[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to /root/nltk_data...
[nltk_data] Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package maxent_ne_chunker to /root/nltk_data...
[nltk_data] Unzipping chunkers/maxent_ne_chunker.zip.
Imports and NLTK downloads done ✅
[nltk_data] Downloading package words to /root/nltk_data...
[nltk_data] Unzipping corpora/words.zip.

Upload Dataset

```
[ ] from google.colab import files
    uploaded = files.upload() # click Choose files and select student_habits_performance.csv
    # after upload, find the filename:
    fnames = list(uploaded.keys())
    print("Uploaded:", fnames)
```

 Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving student_habits_performance.csv to student_habits_performance (1).csv
Uploaded: ['student_habits_performance (1).csv']

Displays dataset shape, first few rows, column types, and missing values.

```
# change filename if different
csv_path = 'student_habits_performance.csv'
```

```
df = pd.read_csv(csv_path)
print("Rows, cols:", df.shape)
display(df.head())
print("\nColumns and dtypes:")
print(df.dtypes)
print("\nMissing values per column:")
print(df.isna().sum())
```

```
Rows, cols: (1000, 16)
```

	student_id	age	gender	study_hours_per_day	social_media_hours	netflix_hours	part_time_job	attendance_percentage	sleep_hours	diet_quality	exercise_frequency	parental_education_level	internet_quality	mental_health_rating	extracurricular_participation	exam_score
0	S1000	23	Female	0.0	1.2	1.1	No	85.0	8.0	Fair	6	Master	Average	8	Yes	55.2
1	S1001	20	Female	6.9	2.8	2.3	No	97.3	4.6	Good	6	High School	Average	8	No	100.0
2	S1002	21	Male	1.4	3.1	1.3	No	94.8	8.0	Poor	1	High School	Poor	1	No	34.3
3	S1003	23	Female	1.0	3.9	1.0	No	71.0	9.2	Poor	4	Master	Good	1	Yes	28.8
4	S1004	19	Female	5.0	4.4	0.5	No	90.9	4.9	Fair	3	Master	Good	1	No	66.4

Columns and dtypes:

student_id	object
age	int64
gender	object
study_hours_per_day	float64
social_media_hours	float64
netflix_hours	float64
part_time_job	object
attendance_percentage	float64
sleep_hours	float64
diet_quality	object
exercise_frequency	int64
parental_education_level	object
internet_quality	object
mental_health_rating	int64
extracurricular_participation	object
exam_score	float64
dtype:	object

Missing values per column:

student_id	0
age	0
gender	0
study_hours_per_day	0
social_media_hours	0
netflix_hours	0
part_time_job	0
attendance_percentage	0
sleep_hours	0
diet_quality	0
exercise_frequency	0
parental_education_level	91
internet_quality	0
mental_health_rating	0
extracurricular_participation	0
exam_score	0
dtype:	int64

Info and Categorical Summary

```
[ ] df.info()
df.describe(include="object") # show categorical values
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 16 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   student_id                            1000 non-null   object
1   age                                    1000 non-null   int64
2   gender                                1000 non-null   object
3   study_hours_per_day                   1000 non-null   float64
4   social_media_hours                    1000 non-null   float64
5   netflix_hours                         1000 non-null   float64
6   part_time_job                         1000 non-null   object
7   attendance_percentage                 1000 non-null   float64
8   sleep_hours                           1000 non-null   float64
9   diet_quality                          1000 non-null   object
10  exercise_frequency                    1000 non-null   int64
11  parental_education_level              909 non-null    object
12  internet_quality                      1000 non-null   object
13  mental_health_rating                  1000 non-null   int64
14  extracurricular_participation         1000 non-null   object
15  exam_score                           1000 non-null   float64
dtypes: float64(6), int64(3), object(7)
memory usage: 125.1+ KB
```

	student_id	gender	part_time_job	diet_quality	parental_education_level	internet_quality	extracurricular_participation
count	1000	1000	1000	1000	909	1000	1000
unique	1000	3	2	3	3	3	2
top	S1999	Female	No	Fair	High School	Good	No
freq	1	481	785	437	392	447	682

Create “text” Column: Builds a **descriptive sentence** for each student by joining columns (age, gender, diet, exercise, internet, mental health).

```
[ ] df['text'] = (
    "Student aged " + df['age'].astype(str) +
    " is " + df['gender'].astype(str) +
    ", follows a " + df['diet_quality'].astype(str) + " diet, " +
    "exercises " + df['exercise_frequency'].astype(str) +
    ", has internet quality rated " + df['internet_quality'].astype(str) +
    ", and reports mental health as " + df['mental_health_rating'].astype(str) + "."
)

df[['student_id', 'text']].head(10)
```

	student_id	text
0	S1000	Student aged 23 is Female, follows a Fair diet...
1	S1001	Student aged 20 is Female, follows a Good diet...
2	S1002	Student aged 21 is Male, follows a Poor diet, ...
3	S1003	Student aged 23 is Female, follows a Poor diet...
4	S1004	Student aged 19 is Female, follows a Fair diet...
5	S1005	Student aged 24 is Male, follows a Fair diet, ...
6	S1006	Student aged 21 is Female, follows a Good diet...
7	S1007	Student aged 21 is Female, follows a Fair diet...
8	S1008	Student aged 23 is Female, follows a Good diet...
9	S1009	Student aged 18 is Female, follows a Good diet...

Tokenization (Sentence + Word)

```
[ ] import nltk
nltk.download('punkt')
nltk.download('punkt_tab')

from nltk.tokenize import word_tokenize, sent_tokenize

# Example: take the first student record
sample_text = df['text'][0]
print("Original Text:\n", sample_text)

# Sentence Tokenization
sent_tokens = sent_tokenize(sample_text)
print("\nSentence Tokens:\n", sent_tokens)

# Word Tokenization
word_tokens = word_tokenize(sample_text)
print("\nWord Tokens:\n", word_tokens)
```

[nltk data] Downloading package punkt to /root/nltk_data...
[nltk data] Package punkt is already up-to-date!
[nltk data] Downloading package punkt_tab to /root/nltk_data...
[nltk data] Unzipping tokenizers/punkt_tab.zip.
Original Text:
Student aged 23 is Female, follows a Fair diet, exercises 6, has internet quality rated Average, and reports mental health as 8.
Sentence Tokens:
['Student aged 23 is Female, follows a Fair diet, exercises 6, has internet quality rated Average, and reports mental health as 8.']
Word Tokens:
['Student', 'aged', '23', 'is', 'Female', ',', ' ', 'follows', 'a', 'Fair', 'diet', ',', ' ', 'exercises', '6', ',', ' ', 'has', 'internet', 'quality', 'rated', 'Average', ',', ' ', 'and', 'reports', 'mental', 'health', 'as', '8', '.']

Stopword + Punctuation Removal

Defines a function that:

- Lowercases
- Removes punctuation/numbers/stopwords

Creates:

- clean_tokens → list of cleaned words
- clean_text → string form

Builds frequency distribution of words and plots top 20.

	student_id	text	clean_text
0	S1000	Student aged 23 is Female, follows a Fair diet...	student aged female follows fair diet exercise...
1	S1001	Student aged 20 is Female, follows a Good diet...	student aged female follows good diet exercise...
2	S1002	Student aged 21 is Male, follows a Poor diet, ...	student aged male follows poor diet exercises ...
3	S1003	Student aged 23 is Female, follows a Poor diet...	student aged female follows poor diet exercise...
4	S1004	Student aged 19 is Female, follows a Fair diet...	student aged female follows fair diet exercise...
5	S1005	Student aged 24 is Male, follows a Fair diet, ...	student aged male follows fair diet exercises ...

Top 20 tokens (overall):

student: 1000
aged: 1000
follows: 1000
diet: 1000
exercises: 1000
internet: 1000
quality: 1000
rated: 1000
reports: 1000
mental: 1000
health: 1000
good: 825
female: 481
male: 477
fair: 437
average: 391
poor: 347

Creates a binary label:

- 1 = above median mental health
- 0 = below median mental health

```
[ ] median_val = df['mental_health_rating'].median()
df['label'] = (df['mental_health_rating'] > median_val).astype(int)
print(df[['mental_health_rating', 'label']].head(10))
```

	mental_health_rating	label
0	8	1
1	8	1
2	1	0
3	1	0
4	1	0
5	4	0
6	4	0
7	8	1
8	1	0
9	10	1

Basic Cleaning

Cleans text:

- Lowercase
- Remove numbers & punctuation
- Collapse whitespace

```

def basic_clean(text):
    """
    Clean text by:
    - Lowercasing
    - Removing numbers & punctuation
    - Stripping extra spaces
    """
    # Lowercase
    text = text.lower()
    # Remove numbers and punctuation
    text = re.sub(r'^a-z\s', '', text)
    # Remove extra spaces
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply cleaning function
df['clean_text'] = df['text'].astype(str).apply(basic_clean)

print("✅ Example cleaned texts (first 6 rows):")
print(df[['student_id', 'text', 'clean_text']].head(6))

```

```

✅ Example cleaned texts (first 6 rows):

```

	student_id	text \	clean_text
0	S1000	Student aged 23 is Female, follows a Fair diet...	0 student aged is female follows a fair diet exe...
1	S1001	Student aged 20 is Female, follows a Good diet...	1 student aged is female follows a good diet exe...
2	S1002	Student aged 21 is Male, follows a Poor diet, ...	2 student aged is male follows a poor diet exerc...
3	S1003	Student aged 23 is Female, follows a Poor diet...	3 student aged is female follows a poor diet exe...
4	S1004	Student aged 19 is Female, follows a Fair diet...	4 student aged is female follows a fair diet exe...
5	S1005	Student aged 24 is Male, follows a Fair diet, ...	5 student aged is male follows a fair diet exerc...

Tokenization

3 ways of tokenizing:

- Simple .split()
- NLTK tokenizer
- spaCy tokenizer

```

♦ Example cleaned text:
student aged is female follows a fair diet exercises has internet quality rated average and reports mental health as

♦ Simple split tokens: ['student', 'aged', 'is', 'female', 'follows', 'a', 'fair', 'diet', 'exercises', 'has', 'internet', 'quality', 'rated', 'average', 'and', 'reports', 'mental', 'health', 'as']

♦ NLTK tokens: ['student', 'aged', 'is', 'female', 'follows', 'a', 'fair', 'diet', 'exercises', 'has', 'internet', 'quality', 'rated', 'average', 'and', 'reports', 'mental', 'health', 'as']

♦ spaCy tokens: ['student', 'aged', 'is', 'female', 'follows', 'a', 'fair', 'diet', 'exercises', 'has', 'internet', 'quality', 'rated', 'average', 'and', 'reports', 'mental', 'health', 'as']

```

Stopword Removal + Frequency

- Removes stopwords.
- Creates clean_text_no_stop.
- Shows top 20 most frequent words after stopwords removal

```

♦ Example without stopwords:
student aged female follows fair diet exercises internet quality rated average reports mental health

♦ Top 20 tokens after stopword removal:
('student', 1000), ('aged', 1000), ('follows', 1000), ('diet', 1000), ('exercises', 1000), ('internet', 1000), ('quality', 1000), ('rated', 1000), ('reports', 1000), ('mental', 1000), ('health', 1000), ('good', 825), ('female', 481).
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data] Package stopwords is already up-to-date!

```

Stemming & Lemmatization

- Uses:
 - PorterStemmer
 - WordNetLemmatizer
- Compares stemmed vs lemmatized text.

```

♦ Original (no stopwords):
student aged female follows fair diet exercises internet quality rated average reports mental health

♦ Stemmed:
student age femal follow fair diet exercis internet qualiti rate averag report mental health

♦ Lemmatized:
student aged female follows fair diet exercise internet quality rated average report mental health

```

Vectorization

- CountVectorizer → converts text to bag-of-words counts.
- TfidfVectorizer → converts text to TF-IDF features.
- Limits features to top 500.

```

♦ CountVectorizer Shape: (1000, 17)
♦ First 10 Features (Count): ['aged' 'average' 'diet' 'exercise' 'fair' 'female' 'follows' 'good'
'health' 'internet']

♦ TF-IDF Shape: (1000, 17)
♦ First 10 Features (TF-IDF): ['aged' 'average' 'diet' 'exercise' 'fair' 'female' 'follows' 'good'
'health' 'internet']

```

Train/Test Split

- Splits dataset into 80% train, 20% test.
- Stratifies so both classes have balanced representation.

```

# ✖ Cell L: Train/Test Split

from sklearn.model_selection import train_test_split

# Features = TF-IDF vectors
X = X_tfidf
# Labels = mental health class (0 or 1)
y = df['label']

# Split into train (80%) and test (20%)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(" ♦ Training set shape:", X_train.shape)
print(" ♦ Test set shape:", X_test.shape)
print(" ♦ Training labels distribution:\n", y_train.value_counts())
print(" ♦ Test labels distribution:\n", y_test.value_counts())

♦ Training set shape: (800, 17)
♦ Test set shape: (200, 17)
♦ Training labels distribution:
label
0    408
1    392
Name: count, dtype: int64
♦ Test labels distribution:
label
0    102
1     98
Name: count, dtype: int64

```

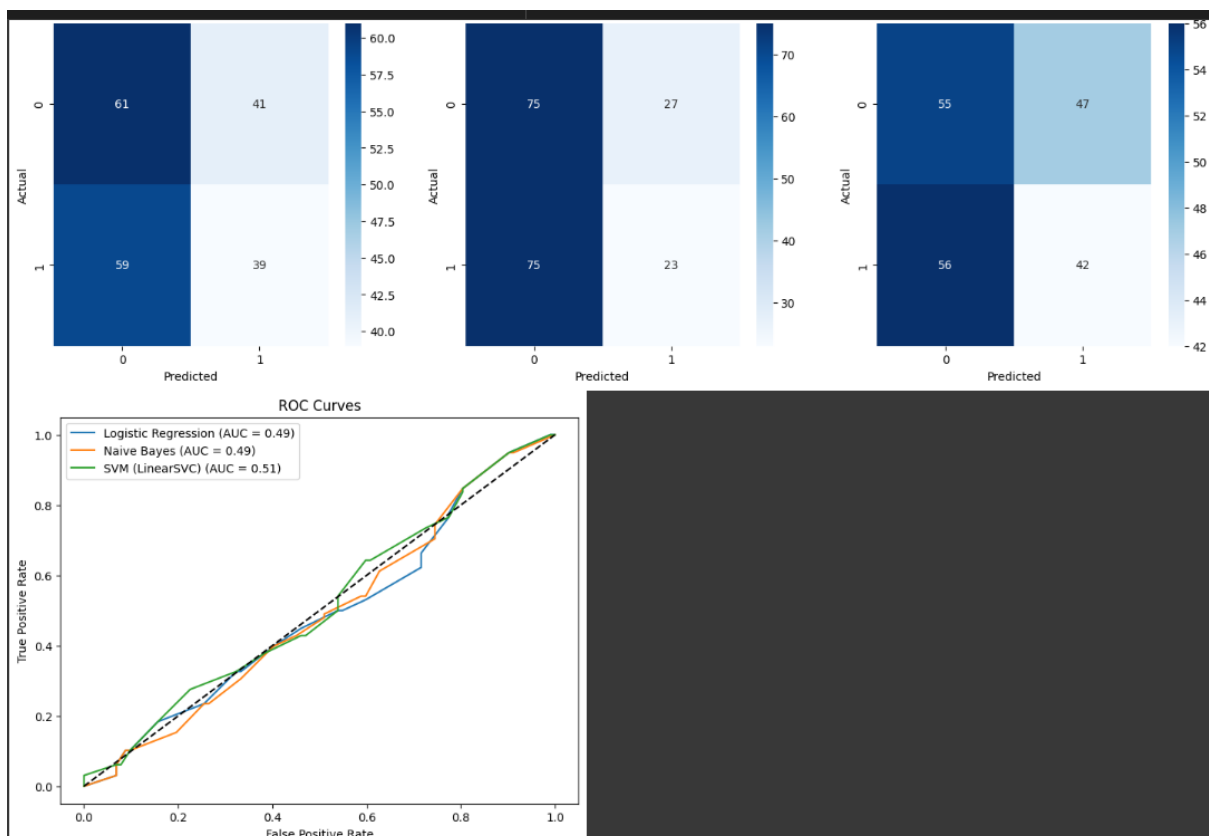
Train Multiple Models

- Trains 3 classifiers:
 - Logistic Regression
 - Naive Bayes
 - Linear SVM
- Evaluates with Accuracy, Precision, Recall, F1.

	Model	Accuracy	Precision	Recall	F1 Score
0	Logistic Regression	0.500	0.48750	0.397959	0.438202
1	Naive Bayes	0.490	0.46000	0.234694	0.310811
2	SVM (LinearSVC)	0.485	0.47191	0.428571	0.449198

Confusion Matrix & ROC Curves

- Plots confusion matrices for each model.
- Draws ROC curves with AUC scores.



POS Tagging & Extract Nouns

	student_id	nouns
0	S1000	[student, quality, reports, health]
1	S1001	[student, diet, exercises, quality, reports, h...]
2	S1002	[student, diet, quality, reports, health]
3	S1003	[student, diet, quality, reports, health]
4	S1004	[student, quality, reports, health]

Named Entity Recognition

- Uses `nlk.ne_chunk` for Named Entity Recognition (NER).
- Displays chunk tree with POS + NER tags.

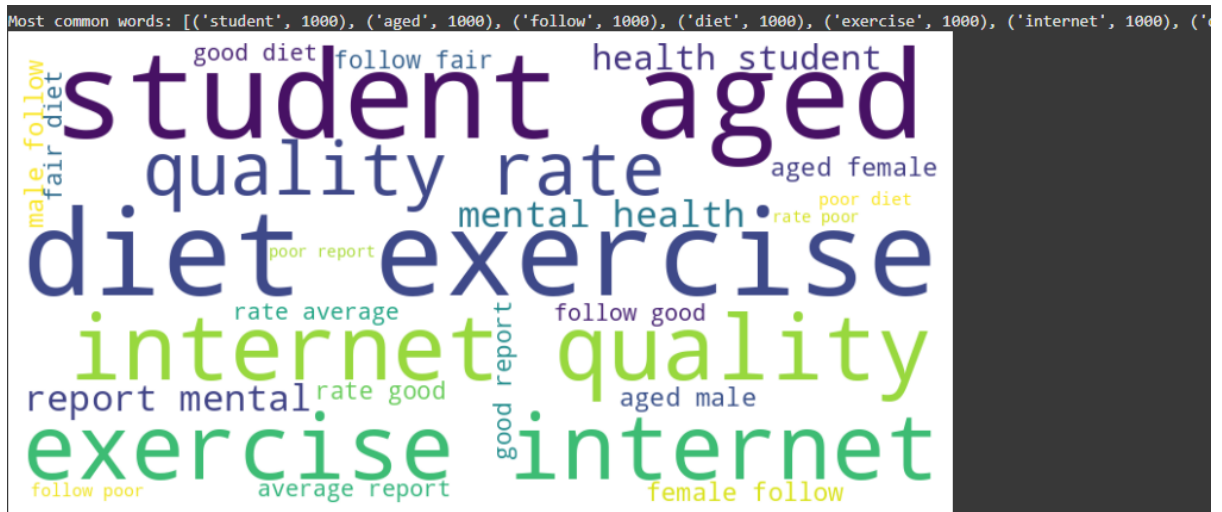
```
POS Tags: [('student', 'NN'), ('aged', 'VBN'), ('is', 'VBZ'), ('female', 'JJ'), ('follows', 'VBZ'), ('a', 'DT'), ('fair', 'JJ'), ('diet', 'NN'), ('exercises', 'VBZ'), ('has', 'VBZ'), ('internet', 'NN'), ('quality', 'NN'), ('rated', 'VBN'), ('average', 'JJ'), ('and', 'CC'), ('reports', 'NNS'), ('mental', 'JJ'), ('health', 'NN'), ('as', 'IN')]
```

NER / Chunk Tree:

```
(S
  student/NN
  aged/VBN
  is/VBZ
  female/JJ
  follows/VBZ
  a/DT
  fair/JJ
  diet/JJ
  exercises/VBZ
  has/VBZ
  internet/VBN
  quality/NN
  rated/VBN
  average/JJ
  and/CC
  reports/NNS
  mental/JJ
  health/NN
  as/IN)
```

WordCloud

- Builds a word frequency dictionary.
- Generates and plots a word cloud of most frequent words



Sentiment Analysis

- Uses NLTK VADER sentiment analyzer.
- Assigns:
 - Positive

- Negative
- Neutral labels.

```
# Download VADER lexicon
nltk.download("vader_lexicon")

# Initialize analyzer
sia = SentimentIntensityAnalyzer()

# Apply sentiment scores
df["sentiment_score"] = df["clean_text"].apply(lambda x: sia.polarity_scores(x)["compound"])

# Label sentiments
df["sentiment_label"] = df["sentiment_score"].apply(
    lambda x: "Positive" if x > 0.05 else ("Negative" if x < -0.05 else "Neutral")
)

# Show sample
print(df[["clean_text", "sentiment_score", "sentiment_label"]].head())
```

[nltk_data] Downloading package vader_lexicon to /root/nltk_data...

	clean_text	sentiment_score \
0	student aged female follow fair diet exercise ...	0.3182
1	student aged female follow good diet exercise ...	0.4404
2	student aged male follow poor diet exercise in...	-0.7351
3	student aged female follow poor diet exercise ...	-0.0516
4	student aged female follow fair diet exercise ...	0.6369

	sentiment_label
0	Positive
1	Positive
2	Negative
3	Negative
4	Positive

Language Detection on Dataset

```
# Apply language detection safely (with try/except)
def safe_detect(text):
    try:
        return detect(str(text))
    except:
        return "unknown"

# Apply to your dataset column (replace 'clean_text' with the right column name)
df["language"] = df["clean_text"].apply(safe_detect)

# Check a few samples
print(df[["clean_text", "language"]].head(10))
```

	clean_text	language
0	student aged female follow fair diet exercise ...	en
1	student aged female follow good diet exercise ...	en
2	student aged male follow poor diet exercise in...	en
3	student aged female follow poor diet exercise ...	en
4	student aged female follow fair diet exercise ...	en
5	student aged male follow fair diet exercise in...	en
6	student aged female follow good diet exercise ...	en
7	student aged female follow fair diet exercise ...	en
8	student aged female follow good diet exercise ...	en
9	student aged female follow good diet exercise ...	en

Small English+Hindi Sample Dataset

Detect Language of Mixed Dataset

```
from langdetect import detect

def safe_detect(text):
    try:
        return detect(str(text))
    except:
        return "unknown"

df_lang["language"] = df_lang["text"].apply(safe_detect)
print(df_lang)
```

	text	language
0	I use my phone daily for studying.	en
1	मुझे मोबाइल पर गेम खेलना पसंद है।	hi
2	Online classes are very useful.	en
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	hi
4	I like watching YouTube for learning.	en
5	क्या तुमने आज नया लेक्चर देखा?	mr

Cleaning for Multilingual Text

	text	cleaned_text
0	I use my phone daily for studying.	i use my phone daily for studying
1	मुझे मोबाइल पर गेम खेलना पसंद है।	मुझे मोबाइल पर गेम खेलना पसंद है।
2	Online classes are very useful.	online classes are very useful
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	मैं रोज़ घंटे मोबाइल पर पढ़ाई करता हूँ।
4	I like watching YouTube for learning.	i like watching youtube for learning
5	क्या तुमने आज नया लेक्चर देखा?	क्या तुमने आज नया लेक्चर देखा

Tokenization Functions

- Defines sentence, word, char, sub-word tokenizers.
- Removes stopwords (both English and Hindi).

	text	cleaned_text	sent_tokens	word_tokens	char_tokens	subword_tokens	filtered_tokens
0	I use my phone daily for studying.	i use my phone daily for studying	[i use my phone daily for studying]	[i, use, my, phone, daily, for, studying]	[I, , u, s, e, , m, y, , p, h, o, n, e, , ...]	[i, use, my, phone, daily, for, studying]	[use, phone, daily, studying]
1	मुझे मोबाइल पर गेम खेलना पसंद है।	मुझे मोबाइल पर गेम खेलना पसंद है।	[मुझे मोबाइल पर गेम खेलना पसंद है।]	[मुझे, मोबाइल, पर, गेम, खेलना, पसंद, है।]	[म, ु, झे, , म, ो, ब, ो, इ, त, , प, र, , ...]	[म, ुझे, म, ोबाइल, पर, गे, ेम, खे, ेलना, पस, ंद, ...]	[मुझे, मोबाइल, गेम, खेलना, पसंद, है।]
2	Online classes are very useful.	online classes are very useful	[online classes are very useful]	[online, classes, are, very, useful]	[o, n, l, i, n, e, , c, l, a, s, s, e, s, , ...]	[online, classes, are, very, useful]	[online, classes, useful]
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	मैं रोज़ घंटे मोबाइल पर पढ़ाई करता हूँ।	[मैं रोज़ घंटे मोबाइल पर पढ़ाई करता हूँ।]	[मैं, रोज़, घंटे, मोबाइल, पर, पढ़ाई, करता, हूँ।]	[म, ैं, ो, ो, , र, ो, ज़, ो, , घ, ं, ट, े, , ...]	[म, ैं, र, ोज़, घ, ंटे, म, ोबाइल, पर, पढ़, ोई, ...]	[रोज़, घंटे, मोबाइल, पढ़ाई, करता, हूँ।]
4	I like watching YouTube for learning.	i like watching youtube for learning	[i like watching youtube for learning]	[i, like, watching, youtube, for, learning]	[i, , l, i, k, e, , w, a, t, c, h, i, n, g, ...]	[i, like, watching, youtube, for, learning]	[like, watching, youtube, learning]
5	क्या तुमने आज नया लेक्चर देखा?	क्या तुमने आज नया लेक्चर देखा	[क्या तुमने आज नया लेक्चर देखा]	[क्या, तुमने, आज, नया, लेक्चर, देखा]	[क, ्या, , तु, म, े, , अ, ज, , न, े, , ...]	[क, ्या, तु, ुमने, आज, नय, ो, त्, ेक्चर, द, ेखा]	[तुमने, आज, नया, लेक्चर, देखा]

Multilingual Tokenization & Frequency

- Applies cleaning + tokenization.
- Builds frequency distributions.
- Shows top words for each sentence.

	text	cleaned_text	tokens_no_stop	top_words
0	I use my phone daily for studying.	i use my phone daily for studying	[use, phone, daily, studying]	[(use, 1), (phone, 1), (daily, 1), (studying, 1)]
1	मुझे मोबाइल पर गेम खेलना पसंद है।	मुझे मोबाइल पर गेम खेलना पसंद है।	[मुझे, मोबाइल, गेम, खेलना, पसंद, है।]	[(मुझे, 1), (मोबाइल, 1), (गेम, 1), (खेलना, 1), ...]
2	Online classes are very useful.	online classes are very useful	[online, classes, useful]	[(online, 1), (classes, 1), (useful, 1)]
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	मैं रोज़ घंटे मोबाइल पर पढ़ाई करता हूँ।	[रोज़, घंटे, मोबाइल, पढ़ाई, करता, हूँ।]	[(रोज़, 1), (घंटे, 1), (मोबाइल, 1), (पढ़ाई, 1), ...]
4	I like watching YouTube for learning.	i like watching youtube for learning	[like, watching, youtube, learning]	[(like, 1), (watching, 1), (youtube, 1), (lear...
5	क्या तुमने आज नया लेक्चर देखा?	क्या तुमने आज नया लेक्चर देखा	[तुमने, आज, नया, लेक्चर, देखा]	[(तुमने, 1), (आज, 1), (नया, 1), (लेक्चर, 1), (...]

Morphological Analysis

- Sets up stemmers and lemmatizer.
- Porter stemmer, Snowball stemmer, WordNet lemmatizer.

index	text	tokens_no_stop	stem_lemma
0	I use my phone daily for studying.	use.phone.daily.studying	{'porter': ['use', 'phone', 'dail', 'studi'], 'snowball': ['use', 'phone', 'dail', 'studi'], 'lemmatized': ['use', 'phone', 'daily', 'studying']}
1	मुझे मोबाइल पर गेम खेलना पसंद है।	मुझे.मोबाइल.गेम.खेलना.पसंद.है।	{'porter': ['मुझे', 'मोबाइल', 'गेम', 'खेलना', 'पसंद', 'है।'], 'snowball': ['मुझे', 'मोबाइल', 'गेम', 'खेलना', 'पसंद', 'है।'], 'lemmatized': ['मुझे', 'मोबाइल', 'गेम', 'खेलना', 'पसंद', 'है।']}
2	Online classes are very useful.	online.classes.useful	{'porter': ['onlin', 'class', 'use'], 'snowball': ['onlin', 'class', 'use'], 'lemmatized': ['online', 'class', 'useful']}
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	रोज़.घंटे.मोबाइल.पढ़ाई.करता.हूँ।	{'porter': ['रोज़', 'घंटे', 'मोबाइल', 'पढ़ाई', 'करता', 'हूँ।'], 'snowball': ['रोज़', 'घंटे', 'मोबाइल', 'पढ़ाई', 'करता', 'हूँ।'], 'lemmatized': ['रोज़', 'घंटे', 'मोबाइल', 'पढ़ाई', 'करता', 'हूँ।']}
4	I like watching YouTube for learning.	like.watching.youtube.learning	{'porter': ['like', 'watch', 'youtub', 'learn'], 'snowball': ['like', 'watch', 'youtub', 'learn'], 'lemmatized': ['like', 'watching', 'youtube', 'learning']}
5	क्या तुमने आज नया लेक्चर देखा?	तुमने.आज.नया.लेक्चर.देखा	{'porter': ['तुमने', 'आज', 'नया', 'लेक्चर', 'देखा'], 'snowball': ['तुमने', 'आज', 'नया', 'लेक्चर', 'देखा'], 'lemmatized': ['तुमने', 'आज', 'नया', 'लेक्चर', 'देखा']}

Install stanza (for Hindi NLP)

- Installs stanza (Stanford NLP toolkit).
- Will be used for Hindi POS tagging, parsing, NER.

index	text	pos_ner
0	I use my phone daily for studying.	{'pos_tags': [('I', 'NN'), ('use', 'VBP'), ('my', 'PRP\$'), ('phone', 'NN'), ('daily', 'RB'), ('for', 'IN'), ('studying', 'VBG')], 'nouns': ['I', 'phone'], 'entities': []}
1	मूझे मोबाइल पर गेम खेलना पसंद है।	{'pos_tags': [('मुझे', 'PRON'), ('मोबाइल', 'NOUN'), ('पर', 'ADP'), ('गेम', 'NOUN'), ('खेलना', 'VERB'), ('पसंद', 'NOUN'), ('है', 'VERB'), ('।', 'PUNCT')], 'nouns': ['मोबाइल', 'गेम', 'पसंद'], 'entities': []}
2	Online classes are very useful.	{'pos_tags': [('online', 'NN'), ('classes', 'NNS'), ('are', 'VBP'), ('very', 'RB'), ('useful', 'JJ')], 'nouns': ['online', 'classes'], 'entities': []}
3	मैं रोज़ 2 घंटे मोबाइल पर पढ़ाई करता हूँ।	{'pos_tags': [('मैं', 'PRON'), ('रोज़', 'NOUN'), ('घंटे', 'NOUN'), ('मोबाइल', 'NOUN'), ('पर', 'ADP'), ('पढ़ाई', 'NOUN'), ('करता', 'VERB'), ('हूँ', 'AUX'), ('।', 'PUNCT')], 'nouns': ['रोज़', 'घंटे', 'मोबाइल', 'पढ़ाई'], 'entities': []}
4	I like watching YouTube for learning.	{'pos_tags': [('I', 'NNS'), ('like', 'IN'), ('watching', 'VBG'), ('youtube', 'NN'), ('for', 'IN'), ('learning', 'VBG')], 'nouns': ['I', 'youtube'], 'entities': []}
5	क्या तुमने आज नया लेक्चर देखा?	{'pos_tags': [('क्या', 'PRON'), ('तुमने', 'PRON'), ('आज', 'NOUN'), ('नया', 'ADJ'), ('लेक्चर', 'NOUN'), ('देखा', 'VERB')], 'nouns': ['आज', 'लेक्चर'], 'entities': []}